

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code: CSA0309	Course Title: Data Structures
CO Assessed: CO4 – Articulation (BL3)	Assessment: Assessment Tool 3 – Comparative Analysis
Weightage: 30% of CIA	Total Marks: 100
CO4 Statement: Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.	
Student Name: G. Nithya Sree	Reg. No.: 192424103
Section / Batch: 3rd year ATDS	Date: 28/08/2026

Code of Conduct

I G. Nithya Sree (192424103) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Nithya Sree

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

Program Director

Signature: _____

Date: _____

1) Compare Linear Probing and Seperate Chaining for collision handling:-

A collision occurs when two or more keys are mapped to the same hash-table index. Linear probing and seperate chaining are two methods used to resolve collisions.

1) Clustering Effect:-

In linear probing after a collision, the next available position is searched sequentially. This can create primary clustering, where consecutive table positions become occupied.

Index: 0 1 2 3 4 5
- 10 20 30 40 -
← cluster →

In seperate chaining, each index contains a chain of elements having the same hash value.

Index 1 → 10 → 20 → 30.

Index 2 → 15.

Therefore, seperate chaining avoids the primary clustering problem of linear probing.

2) Memory Usage:-

Linear probing stores elements directly in the hash table, so it requires less additional memory.

seperate chaining requires additional nodes and pointers / references for the chains.

3) Performance under high load factor:-

Load factor is $\alpha = \frac{n}{m}$. Where n is the number of elements and m is the table size.

When the load factor becomes high, linear probing becomes slower because empty positions become scarce and clustering increases. Separate chaining handles high load factors better because multiple elements can be stored in the same bucket.

4) Complexity:-

Operation	Linear Probing	Separate chaining
Average search	$O(1)$	$O(1 + \alpha)$
Average Insertion	$O(1)$	$O(1)$
Worst search	$O(n)$	$O(n)$
Space	$O(m)$	$O(m + n)$

5) Separate suitable method for large-scale Applications:-

Separate chaining is generally more suitable for large scale and dynamic applications because it handles high load factors better, avoids primary clustering, and makes deletion easier. However, linear probing is preferable when memory efficiency and cache locality are important and the load factor can be kept low.

Conclusion:-

Linear probing provides better memory efficiency and cache performance, while separate chaining provides better flexibility and high-load performance. Therefore, separate chaining is generally preferred for large, dynamic applications.

Compare Quick sort and Merge sort for sorting a Large Linked List of 10 million Nodes:-

Quick sort and Merge sort are both efficient comparison-based sorting algorithm, but suitability depends on the data structure.

For a linked list containing 10 million nodes, Merge sort is generally preferred.

1) Time Complexity:-

Case	Quick Sort	Merge Sort
Best	$O(n \log n)$	$O(n \log n)$
Average	$O(n \log n)$	$O(n \log n)$
Worst	$O(n^2)$	$O(n \log n)$

Quick sort can become $O(n^2)$ if poor pivots are repeatedly selected. Merge sort consistently provides $O(n \log n)$ performance.

2) Memory Requirements:-

Quick sort:-

Quick sort can be implemented in-place, meaning it does not require another array for sorting elements. However, recursive ~~sort~~ Quick sort requires stack space.

- Average stack: $O(\log n)$.
- Worst case stack: $O(n)$.

Merge Sort:-

For linked list, Merge sort can be implemented by rearranging node pointers, so it does not require an extra array containing all 10 million nodes.

its recursive stack requires $O(\log n)$. Therefore, Merge sort for linked lists can be implemented with $O(\log n)$ auxiliary stack space apart from the nodes themselves.

3) Cache Behaviour:-

Quick sort generally has good cache locality when working on arrays because nearby elements are accessed together. However, linked-list nodes may be scattered in memory. Therefore, Quick sort does not receive the same cache benefits as it does with arrays. Merge sort works particularly well with linked list because it sequentially traverse the list and changes pointers during merging.

Therefore, merge sort is generally more suitable for linked list.

4) Why merge sort is preferred for linked list:-

Merge sort is preferable because:

- 1) It guarantees $O(n \log n)$ time.
- 2) It does not require random access.
- 3) It can split linked lists efficiently using pointers.
- 4) Merging linked list can be done by changing pointers rather than moving data.

Conclusion:-

For a 10-million-node linked list, Merge sort is preferable because linked list do not support efficient random access and merge sort works naturally through sequential traversal and pointer manipulation.

For arrays, Quick sort is often preferred in practice because of its cache efficiency and in-place operation, although merge sort is better when guaranteed $O(n \log n)$ time and stability are required.

Compare Quadratic Probing and Double Hashing:

Both Quadratic probing and Double Hashing are open-addressing techniques used to resolve collisions in hash tables.

1) Clustering Issues:-

(Both Quadratic probing and Double hashing are open addressing techniques used to resolve collisions in hash tables.)^x

Quadratic Probing:-

Instead of checking consecutive positions, quadratic probing uses a quadratic sequence: $h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$. This reduces primary clustering compared with linear probing.

Double Hashing:-

Double hashing uses a second hash function:

$h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$. Different keys generally receive different probing sequences.

Therefore double hashing reduces both primary and secondary clustering more efficiently.

2) Collision Resolution Efficiency:-

Feature	Quadratic Probing	Double Hashing
Primary Clustering	Reduced	Greatly reduced
secondary clustering	Present	Much lower
Probe sequence	Quadratic	Based on second hash
Efficiency	Good	Generally better.

Double hashing usually distributes probes more uniformly throughout the table.

3) Suitability for Dense Hash Tables:-

When the load factor becomes high, collisions become

more frequent. Quadratic probing can suffer from secondary clustering and may not use all table positions depending on the table size and constants.

Therefore: Double hashing is generally more suitable for dense hash tables.

4) Complexity:-

With a good hash function:

- Average search/insertion: $O(1)$.
- Worst - case search/insertion: $O(n)$.
- Space: $O(m)$.

However, actual performance depends strongly on the load factor and quality of the hash functions.

Conclusion:-

Quadratic probing is simpler and reduces primary clusters but it can still suffer from secondary clustering.

Double Hashing provides better distribution of probes and is generally more efficient for dense hash tables.

Therefore, Double Hashing is the better choice when collision distribution and high-load performance are important.

4) Compare Best-Case and Worst-Case time Complexity of sorting Algorithms:-

Time complexity describes how the running time of an algorithm changes as the input size n increases. The best cases represents the most favourable input, while the worst case represent the most unfavourable input.

Comparison of sorting Algorithms:-

Algorithm	Best case	Average Case	Worst case.
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$.
Selection sort	$O(n^2)$.	$O(n^2)$	$O(n^2)$.
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$.
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$.
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

2) Practical Significance:-

Best Case:-

Best-case complexity tells us how efficiently an algorithm can work when the input has favourable characteristics.

For eg: Insertion sort has $O(n)$ best case time when the data is already sorted or nearly sorted.

Worst Case:-

Worst case complexity tells us the maximum amount of time an algorithm may require for a particular input.

3) Impact on Algorithm Selection:-

The choice of sorting algorithm should depend on,

- Input size.
- input order.
- memory availability.
- stability requirements.
- need for predictable performance.

4) Critical Analysis:-

Best-case complexity alone should not be used to select an algorithm because favourable input may not always occur.

Worst-case complexity is especially important for systems where performance must be predictable.

Conclusion:-

Best-case complexity helps understand the potential performance under favourable conditions, while worst-case complexity helps evaluate reliability and performance guarantees. For large and unpredictable datasets, algorithms with a guaranteed $O(n \log n)$ worst case, such as Merge sort and Heap sort.

5) Compare Recursive and iterative implementations of Quick Sort:-

Quick sort can be implemented using either recursive or an iterative approach using an explicit stack.

1) Space Complexity:-

In recursive Quick sort, every partition creates recursive function calls.

Best / Average - $O(\log n)$.

Worst - $O(n)$.

2) Case of implementation:-

Recursive:-

Recursive Quick sort is easier to understand and implement. The function automatically manages the call stack.

Iterative:-

Iterative Quick sort requires manually maintaining a stack. Therefore, it is more complicated than the recursive version.

Risk of stack Overflow:-

Recursive Quick sort can cause stack overflow when partitions becomes highly unbalanced. This can create $O(n)$ recursive calls. Iterative Quick sort avoids the function - call stack overflow problem because it manages it's own stack in the heap.

Conclusion:-

Recursive Quick sort provides simplicity and readability, but it can suffer from stack overflow in the worst case. Iterative Quick sort requires more implementation effort but provides better control over stack usage, making it more suitable when stack limitations are important.

Woj